

Python - NamedTuple

In Python, **tuples** are immutable data structures that can hold a collection of items. We already discussed tuples, its properties and usage in the previous chapters. In this chapter, we will explain **namedtuple()** and its usage in Python.

What is NamedTuple?

In Python, a **namedtuple()** is a subclass of **tuple** that allows you to create tuples with named fields. Meaning, you can access the elements of the tuple using fieldnames instead of just numerical indices. It is part of the **collections** module in Python.

Syntax

```
from collections import namedtuple

# Define a namedtuple
Point = namedtuple('typename', fieldnames )
```

- **Type Name:** The typename is the name of the namedtuple class. It is a string that represents the name of the tuple type.
- **Field Names:** The field names are the names of the elements in the tuple. They are defined as a list of strings.

Create a NamedTuple in Python

To create a namedtuple, you need to import the **namedtuple** function from the **collections** module. Then, you can define a namedtuple by specifying its name and field names.

The following **example** shows how to create a namedtuple called **Point** with fields **x** and **y** –

```
from collections import namedtuple

# Define a namedtuple
Vertex = namedtuple('Vertex', ['x', 'y'])

# Create an instance
v = Vertex(10, 20)

# Access fields
print("Vertex-1:", v.x)
print("Vertex-2:", v.y)
```

The **output** of the above code will be –

```
Vertex-1: 10
Vertex-2: 20
```

Access NamedTuple Fields

We already saw how to access the fields of a namedtuple using dot notation(i.e., using keyname). There are some other ways to access the fields as well –

- **Accessing by Indexing** – You can access the fields using their index, just like a regular tuple.
- **Accessing by keyname** – You can also access the fields using their key names, similar to a dictionary.
- **Accessing Using `getattr()`** – You can use the **`getattr()`** function to access the fields by name.

Example: Accessing by Indexing

In this example, we will access the fields of the namedtuple using their index.



```

from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Access fields by indexing
print("Point-1", p[0])
print("Point-2", p[1])

```

The **output** of the above code will be –

```

Point-1 10
Point-2 20

```

Example: Accessing by keyname

In this example, we will access the fields of the namedtuple using their key names.



```

from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Access fields by keyname
print("Point-1:", p.x)
print("Point-2:", p.y)

```

The **output** of the above code will be –

```

Point-1: 10

```

Point-2: 20

Example: Accessing Using getattr()

In this example, we will access the fields of the namedtuple using the `getattr()` function.

```
from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Access fields using getattr()
print("getattr(p, 'x'):", getattr(p, 'x'))
print("getattr(p, 'y'):", getattr(p, 'y'))
```

The **output** of the above code will be –

```
getattr(p, 'x'): 10
getattr(p, 'y'): 20
```

NamedTuple Methods

Namedtuples come with additional built-in methods to handle common operations. The section below discusses some of these methods with examples.

The `_fields()` method

The **`_fields()`** method is used to access the field names of the namedtuple. It doesn't need any arguments and returns a tuple of the field names.

The following **example** demonstrates how to use the **`_fields()`** method –



```

from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Access fields using _fields()
print("Fields of p:", p._fields)

```

The **output** of the above code will be –

```
Fields of p: ('x', 'y')
```

The `_replace()` method

The **`_replace()`** method is used to create a new instance of the namedtuple with one or more fields replaced with new values. It takes keyword arguments where the keys are the field names and the values are the new values to be assigned.

The following **example** demonstrates how to use the **`_replace()`** method –



```

from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Replace a field value
p2 = p._replace(x=30)

# Access fields
print("p2.x:", p2.x)
print("p2.y:", p2.y)

```

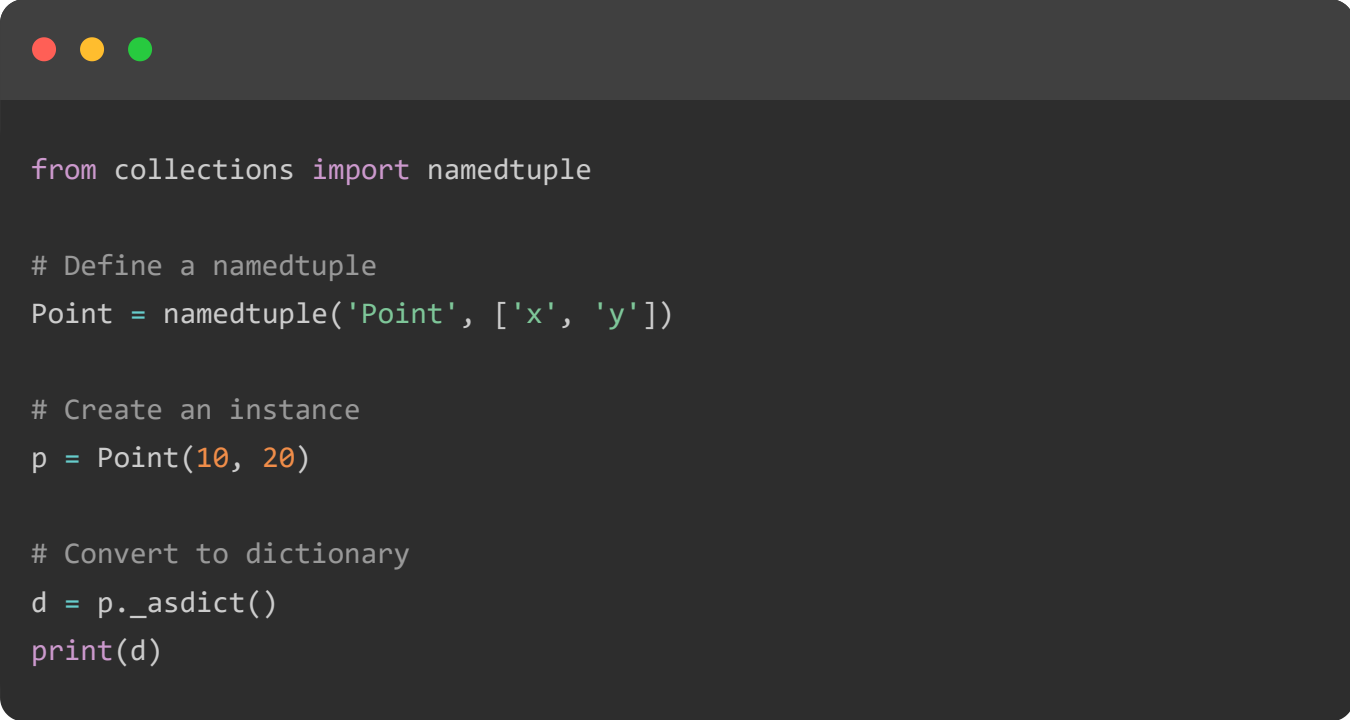
The **output** of the above code will be –

```
p2.x: 30  
p2.y: 20
```

The `_asdict()` method

The **`_asdict()`** method is used to convert the namedtuple into a regular dictionary. This function returns the `OrderedDict()` as constructed from the mapped values of `namedtuple()`.

The following **example** demonstrates how to use the **`_asdict()`** method –



```
from collections import namedtuple  
  
# Define a namedtuple  
Point = namedtuple('Point', ['x', 'y'])  
  
# Create an instance  
p = Point(10, 20)  
  
# Convert to dictionary  
d = p._asdict()  
print(d)
```

The **output** of the above code will be –

```
{'x': 10, 'y': 20}
```

The `_make()` method

The **`_make()`** method is used to create a new instance of the namedtuple from an iterable (like a list or a tuple).

The following **example** demonstrates how to use the **`_make()`** method –



```

from collections import namedtuple

# Define a namedtuple
Point = namedtuple('Point', ['x', 'y'])

# Create an instance
p = Point(10, 20)

# Create a new instance using _make()
p2 = Point._make([30, 40])

# Access fields
print("p2.x:", p2.x)
print("p2.y:", p2.y)

```

The **output** of the above code will be –

```

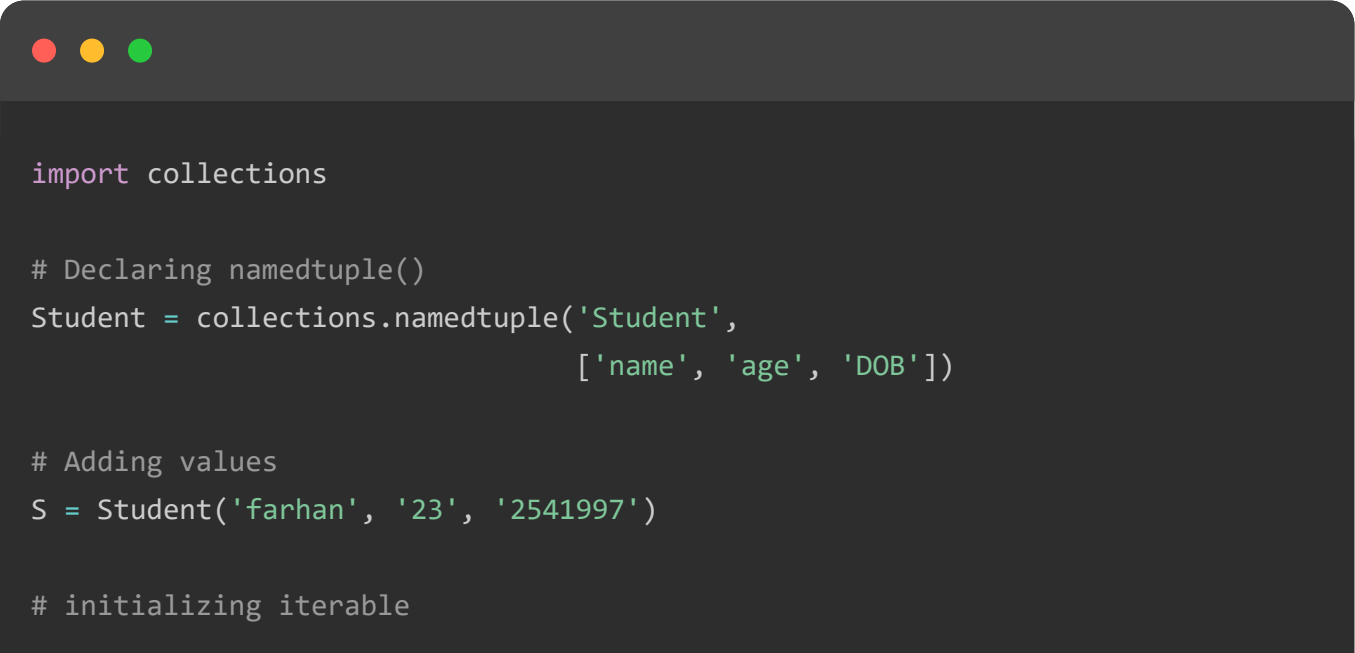
p2.x: 30
p2.y: 40

```

The ** (double star) operator

The ** (double star) operator is used to unpack the fields of the namedtuple into keyword arguments. This can be useful when you want to pass the fields of a namedtuple to a function that accepts keyword arguments.

The following **example** demonstrates how to use the ** operator –



```

import collections

# Declaring namedtuple()
Student = collections.namedtuple('Student',
                                  ['name', 'age', 'DOB'])

# Adding values
S = Student('farhan', '23', '2541997')

# initializing iterable

```

```
li = ['nishu', '19', '411997']

# initializing dict
di = {'name': "ravi", 'age': 24, 'DOB': '1391997'}

# using ** operator to return namedtuple from dictionary
print("The namedtuple instance from dict is : ")
print(Student(**di))
```

The **output** of the above code will be –

```
The namedtuple instance from dict is :
Student(name='ravi', age=24, DOB='1391997')
```

NamedTuple vs Dictionary vs Classes

The main differences between NamedTuple, Dictionary, and Classes are as follows –

Feature	NamedTuple	Dictionary	Class
Syntax	Simple, just like tuple	Key value pairs	More complex, with methods and attributes
Accessing Elements	By name or index	By key	By attribute
Mutability	Immutable	Mutable	Mutable
Memory Efficiency	Most efficient	Less efficient	Least efficient

Conclusion

In this chapter, we have learned about namedtuples in Python, and its differences from dictionaries and classes. A common question arises is that, when to use NamedTuple, Dictionary, or a Class? The **NamedTuple** can be used when you need an immutable object with fixed fields, i.e., if you want features of both tuples and dictionaries. The **Dictionary** can be used when you need a mutable object with dynamic keys. and **Class** is used when you need a complex data structure with methods and behavior.

TOP TUTORIALS

Python Tutorial
Java Tutorial
C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification

Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Chapters ▾

Categories ≡

Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.